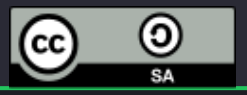
open-testing-agents:*selfhost*

A self-hosted, fully declarative testing lab

Testing the agentic stack

no cloud and no hand-edited state

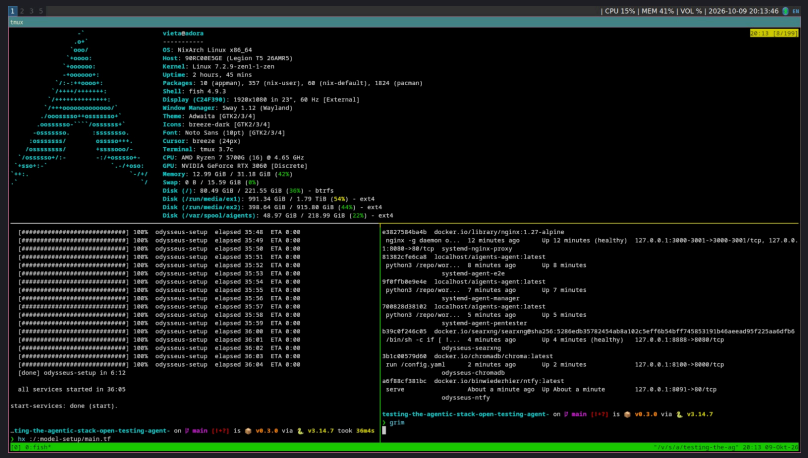
Thanh Viet Nguyen - Vieta



Licensed under CC BY-SA 4.0

Agenda

Motivation
What is this repo?
The three agents
Stack architecture
Models & memory
MCP knowledge bus
Guardrails: Lean4 harness
Everything as code
Interfaces: CLI & Odysseus
Stack in numbers
Status & lessons learned
Thank you to be here
Appendix



Motivation

- Agentic testing usually means **cloud APIs and vendor lock-in**.
- External model providers see your test code and data.
- Brokers (e.g. MQTT) and **hand-edited state** add moving parts.
- Distributed setups (the old iteration used **Raspberry Pis + MQTT**).

- **This stack:** everything on a single machine you control end to end.
- No cloud, no MQTT broker, no hand-edited state.
- Fully declarative from the OS toolchain up.
- GPU inference, vector store, formal guardrails - all local.

What is this repo?

Open-Testing-Agents:*selfhost* is a small but complete **agentic stack**: three AI agents test a real web application - **OWASP Juice Shop** - chain their findings into one report and stream the result to a terminal interface.

Agents 3 CrewAI crews, 3 roles	**Model** 2x llama.cpp, self-hosted	**Memory** Postgres + pgvector	**Bus** small MCP server
SUT Juice Shop via nginx	**Guardrails** Lean4 harness	**Interface** terminal CLI + Odysseus	**IaC** OpenTofu + Dhall + Nix

Video presentation: https://codeberg.org/spifamily/open-testing-agents-abgabe

The three agents

Role	Crew dir	Work	Output
e2e engineer	`build/e2e`	Playwright tests against the SUT	`playwright_test.py` + `previous_output.md`
pentester	`build/pentester`	nmap : nikto : sqlmap : nuclei scans	`previous_output.md`
test manager	`build/manager`	controller: assign, monitor, follow-ups	`report.md` (process + findings + mail)

- All three run as **Podman Quadlet** services and carry the same `custom:aigents_bus` tool -> they exchange findings **peer-to-peer** over the MCP bus.
- The **test manager is the controller**: assigns each case to e2e and/or pentester, monitors their state, dispatches follow-ups and produces the final report.
- Prometheus could receiv mails for the test reports which is not working yet

Stack architecture



Agents also reach the SUT and each other directly; the Lean4 harness validates every run before it executes.

Models & memory

- **phi-4-mini** - agents' brain, `:18080` (llama.cpp, CUDA)
  - 3.8B dense, 128K native ctx, **native tool calling**
  - 65536 ctx total = 16384 per slot, 4 parallel slots
  - all 99 layers offloaded to an **RTX 3060 (12 GB)**
- **phi-mini-moe** - **optional** fast helper, `:18081`
  - 7.6B total / 2.4B active experts, runs on **CPU**
  - condenses shared findings before they enter the primary's context
  - extra models are **optional**: `secondary_model_enabled = false` serves only phi-4-mini, the condense step is skipped - everything degrades gracefully
- **PostgreSQL + pgvector** - reports, tasks, findings, embeddings
  - vector **3072** (matches phi-4-mini hidden size)
  - schema seeded once on an empty data dir
- Both GGUF weights are downloaded by `model-fetch` one-shot profiles
  - phi-4-mini 2.5 GB, phi-mini-moe 5 GB
- One podman-compose stack (`project aigents`) managed by OpenTofu

MCP knowledge bus

- stdlib-only **JSON-RPC 2.0** over HTTP - runs on the host as a systemd **user** service
- SQLite state at `/var/spool/aigents/mcp/mcp.db`
- agents + CLI + Odysseus chat speak the same protocol
- every failure is a tool message, never a broken run

Bus tool	Purpose
`assign_test_case`	manager -> queue a case
`get_next_task` / `list_tasks`	claim + list tasks
`submit_findings` / `get_findings`	knowledge exchange
`start_agent` / `stop_agent` / `get_agent_status`	systemd control
`publish_process` / `publish_note`	Odysseus docs + notes
`mail_report`	final report as e-mail

Guardrails: Lean4 harness

- **AgentHarness.lean** - formal types for
  - action boundaries + tool pre-conditions
  - worker state transitions
  - the **single SUT** every role is scoped to
- Lean 4 port of the **Agda** original (`skills/agda` kept for traceability)
- Role-aware **WellTuned** envelopes (`queue_size, dedupe_cap, crew_timeout, steps, step_cost`)
- Every cycle the worker runs `lake env lean --run Main.lean <role> …`
- Verdict `accepted` -> crew runs; `rejected` -> logged to Postgres
- **Off-target is rejected**: any SUT URL that is not Juice Shop fails the check

Everything as code

Dhall

- crew JSON-first config is **typed Dhall**
- `just crews` -> `.dhall/` -> `build/<role>/`
- safe by default (total functional language)

OpenTofu

- root `deploy/` module composes 5 child modules
- `service_state=running|stopped`, `run_interval`, ...
- only changed triggers restart (fast code round-trips)

Nix + just

- `flake.nix` dev shell: tofu, dhall, lean, just, ...
- `just deploy` / `down` / `status` / `pace 60` / `start-fast`
- no NixOS required - pure nixpkgs toolchain

Interfaces: CLI & Odysseus

`tm_cli`

- REPL with the test-manager persona (role/goal/backstory)
- `/status` `/case` `/tasks` `/findings` `/process`
- `/watch` - real-time dashboard (agents, tasks, findings)
- `/save` `/latest` - transcripts to Postgres

Odysseus

- self-hosted AI workspace (forked, Lean4 modifications), `:7000`
- chat with the local models + **Test Manager** character
- **Notes** panel: per-role `test-results` notes (Keep-style)
- **Documents**: live testing-process markdown, published in place
- e-mail delivery of final reports (SMTP/IMAP)

Stack in numbers

- 3 agents : 2 models : 1 SUT
- 5 OpenTofu modules
- 7.5 GB of GGUF weights
- 64K context : 3072 embedding dim
- default run interval: 900 s

Service	Port	Owner
juice shop (SUT via nginx)	8080	sut-setup
juice shop (direct)	3000	sut-setup
grafana	3001	sut-setup
phi-4-mini	18080	model-setup
phi-mini-moe	18081	model-setup
postgres + pgvector	15432	model-setup
MCP bus	8765	mcp-setup
Odysseus UI : SearXNG : ChromaDB : ntfy	7000 : 8888 : 8100 : 8091	odysseus-setup

Status & lessons learned

- **Alpha phase** - split from MQTT + Raspberry Pis into a single self-hosted host, ready for future projects
- **Future projects:** more optimized (open-weight) models that run on any system - 2 GB RAM and 4 CPU cores
- Qdrant/brokers dropped -> Postgres + pgvector + a direct MCP bus
- A local agentic testing setup is achievable **for everyone** (no cloud, no vendor lock-in)
- Proof assistants (Rocq, Lean4, Agda) are not only for mathematicians - they guard real agent workflows
- **Less can be more:** fewer moving parts (no brokers, no external APIs) -> fewer failure modes

Open weights + open source -> digital sovereignty

- The agents' brain runs on **open-weight** models: Phi-4-mini and Phi-mini-MoE as local GGUF via llama.cpp
- No proprietary cloud API, no hidden inference.
- **All data stays on the machine**: test cases, findings, prompts and reports never leave the host.
- The toolchain is fully open source: llama.cpp , CrewAI : OpenTofu , Podman , Nix , Lean4 , Dhall .
- Open weights + open code mean the stack can be **independently verified, audited, forked and improved** - a real building block for digital sovereignty.

Thank you to be here

Motivation	Guardrails	Reality
self-host, no cloud	Lean4 harness	Still work in progress



Thanks for listening - happy to take questions and exchange ideas. ❤️

Appendix

- https://github.com/yVieta/testing-the-agentic-stack-open-testing-agent-
- https://codeberg.org/spifamily/ (talk video)
- https://github.com/yVieta/odysseus (fork with Lean4 modifications)
- `docs/USER_MANUAL_TEST_CASES.md` - step-by-step test-case manual
- https://owasp.org/www-project-juice-shop/
- https://www.crewai.com/
- https://github.com/ggml-org/llama.cpp
- https://opentofu.org/ : https://dhall-lang.org/
- https://lean-lang.org/

Sources - `resources/Papers.md`

Models
- https://github.com/microsoft/PhiCookBook
- https://arxiv.org/abs/2412.08905

Fine tuning
- https://odr.chalmers.se/bitstreams/f68aff4a-3865-41b0-ac54-2ce3d917e783/download
- https://arxiv.org/html/2408.13296v1
- https://github.com/Curated-Awesome-Lists/awesome-llms-fine-tuning

Others
- https://arxiv.org/html/2501.00217v1
- https://arxiv.org/html/2407.03203

Offtopic (still inspirational)
- https://arxiv.org/pdf/2510.12787

Extension (WIP)
- https://github.com/formalverification/agda-native-air